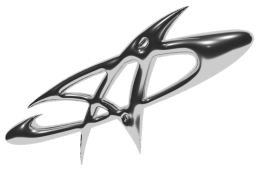
Week 05 · Add & Norm, Feed Forward, Encoder Block

조각들을 모아 하나의 Encoder Block으로

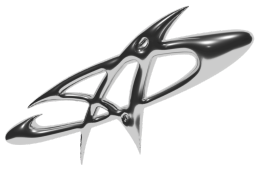
Residual Connection과 Layer Normalization으로 깊은 네트워크를 안정적으로 학습시키고, Feed Forward Network까지 더해 Encoder Block 하나를 완성합니다.



이 자료의 모든 저작권은 조치연에게 있습니다 · 무단 도용 금지

이론 세션

Encoder Block 하나를 완성하기 위해 필요한 것들

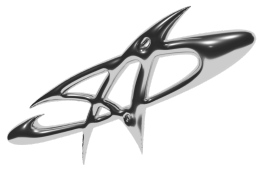


이 자료의 모든 저작권은 조치연에게 있습니다 · 무단 도용 금지

지금까지 만든 조각들

우리는 이미 절반을 만들었다

- **3주차** — Scaled Dot-Product Attention (Q, K, V로 문맥을 섞는 연산)
- **4주차** — Multi-Head Attention (여러 관점 병렬 처리) + Positional Encoding (순서 정보)
- 오늘은 이 조각들을 **안정적으로 학습되는 하나의 블록**으로 조립합니다
- 필요한 나머지 두 가지: **Residual Connection**과 **Layer Normalization**, 그리고 **Feed Forward Network**

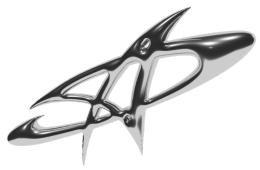


이 자료의 모든 저작권은 조치연에게 있습니다 · 무단 도용 금지

Residual Connection (잔차 연결)

입력을 출력에 그대로 더해준다

- 수식은 아주 단순: **output = x + Sublayer(x)** (Sublayer는 Attention이나 FFN)
- 층이 깊어질수록 기울기가 소실되는 문제를 완화 — “그래도 최소한 x는 그대로 전달된다”는 지름길(shortcut) 확보
- Sublayer가 “아무것도 안 배워도” 최소한 항등함수(identity)는 보장됨 — 학습이 훨씬 안정적
- ResNet(이미지 분야)에서 먼저 검증된 아이디어를 Transformer가 그대로 차용한 것



이 자료의 모든 저작권은 조치연에게 있습니다 · 무단 도용 금지

Layer Normalization

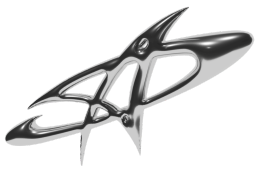
“한 샘플, 한 위치” 안에서 평균 0 · 분산 1로

$$LN(x) = \gamma \cdot \frac{x - \mu}{\sqrt{\sigma^2 + \varepsilon}} + \beta$$

μ, σ^2 — 하나의 토큰 벡터(feature 차원) 안에서 계산한 평균과 분산

γ, β — 학습되는 스케일/시프트 파라미터 (정규화 후 다시 표현력을 되찾아줌)

ε — 분모가 0이 되는 것을 막는 아주 작은 상수



이 자료의 모든 저작권은 조치연에게 있습니다 · 무단 도용 금지

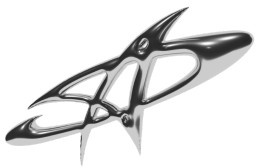
왜 NLP는 Batch Norm 대신 Layer Norm을 쓰는가

Batch Norm

- **배치(batch) 방향**으로 평균/분산 계산
- 배치 크기가 작으면 통계가 불안정
- 문장마다 길이가 달라 패딩 위치까지 섞이면 왜곡됨

Layer Norm

- **한 토큰의 feature 방향**으로 평균/분산 계산
- 배치 크기, 문장 길이와 무관하게 항상 안정적
- 각 토큰이 독립적으로 정규화되어 가변 길이 시퀀스에 자연스러움

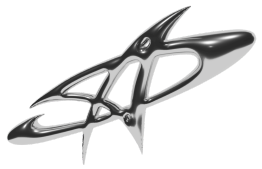


이 자료의 모든 저작권은 조치연에게 있습니다 · 무단 도용 금지

Position-wise Feed Forward Network

각 단어 위치마다 **독립적으로** 적용되는 2층 MLP

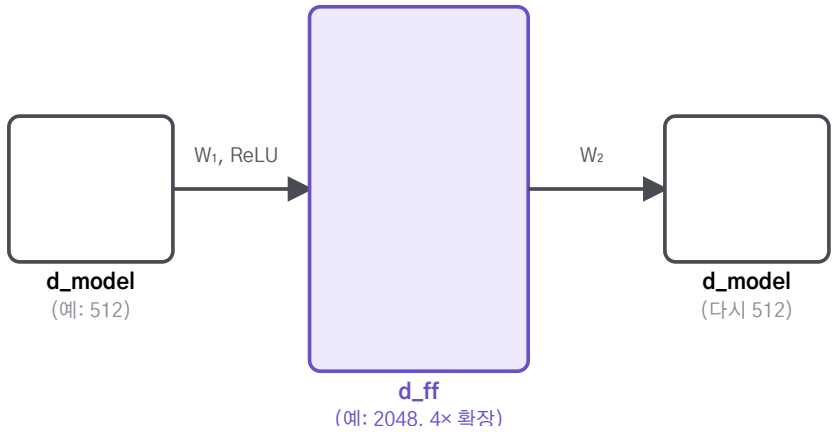
- $FFN(x) = \max(0, xW_1 + b_1)\, W_2 + b_2$ — Linear → ReLU → Linear
- “position-wise”라는 이름대로, **같은 FFN을 모든 위치(단어)에 각각 독립적으로** 적용 (위치끼리 정보 교환은 없음)
- Attention이 **“단어 사이”** 정보를 섞는 역할이라면, FFN은 **“섞인 정보를 각 위치에서 한 번 더 가공”**하는 역할
- 보통 은닉 차원을 d_model보다 훨씬 크게(예: 512 → 2048) 키웠다가 다시 줄여서 표현력을 확보



이 자료의 모든 저작권은 조치연에게 있습니다 · 무단 도용 금지

FFN, 차원으로 보면

좁게 들어와서, 넓어졌다가, 다시 좁아진다



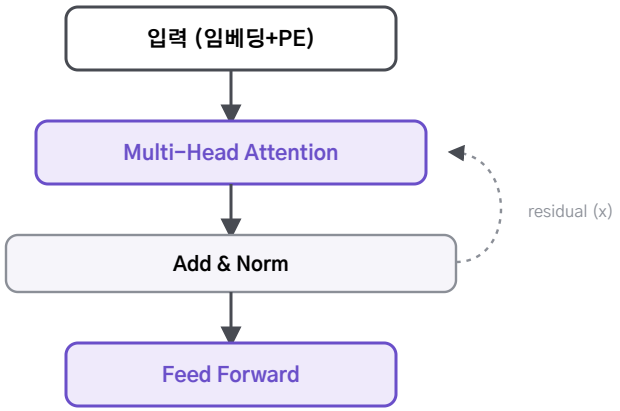
넓은 공간(d_ff)에서 더 복잡한
패턴을 표현한 뒤, 원래 크기로
되돌려 다음 층과 shape을 맞춥니다.

박스의 높이가 차원 크기를 나타냅니다 — d_model → d_ff로 넓혔다가, 다시 d_model로 줄이는 "병목 반대(expand-contract)" 구조입니다.

이 자료의 모든 저작권은 조치연에게 있습니다 · 무단 도용 금지

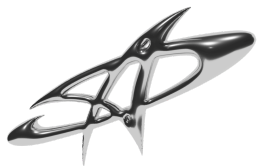
Encoder Block 조립도

Attention과 FFN, 각각에 Add&Norm을 붙이면



두 번째 Add&Norm은
지면상 생략 — 코드에서
동일하게 한 번 더 반복

Encoder Block = MHA + Add&Norm + FFN + Add&Norm. 이 블록을 N번 쌓은 것이 Encoder 전체입니다.

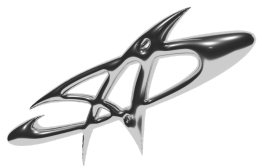


이 자료의 모든 저작권은 조치연에게 있습니다 · 무단 도용 금지

Dropout은 어디에 넣는가

과적합을 막는 마지막 디테일

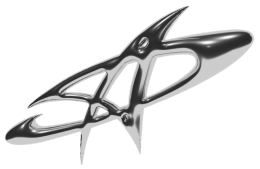
- **Attention weights** 직후, **FFN 출력** 직후, 그리고 **Add(잔차 연결) 직전**에 Dropout을 적용하는 것이 관례
- Embedding + Positional Encoding을 더한 직후에도 Dropout 하나 추가
- 학습 초반에 과적합 걱정이 없다면 0.1 정도의 작은 값으로 시작해도 충분



이 자료의 모든 저작권은 조치연에게 있습니다 · 무단 도용 금지

코딩 세션

EncoderLayer 구현 → Encoder Stack까지



이 자료의 모든 저작권은 조치연에게 있습니다 · 무단 도용 금지

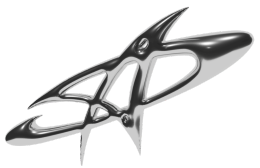
Position-wise Feed Forward 구현

Linear → ReLU → Linear

01_feedforward.py

```
class PositionwiseFeedForward(nn.Module):
    def __init__(self, d_model, d_ff, dropout=0.1):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(d_model, d_ff),
            nn.ReLU(),
            nn.Dropout(dropout),
            nn.Linear(d_ff, d_model),
        )

    def forward(self, x):
        return self.net(x)   # [B, T, d_model] → 동일 shape
```

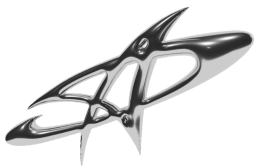


이 자료의 모든 저작권은 조치연에게 있습니다 · 무단 도용 금지

MHA, FFN, LayerNorm 두 벌, Dropout

02_encoder_layer_init.py

```
class EncoderLayer(nn.Module):
    def __init__(self, d_model, num_heads, d_ff, dropout=0.1):
        super().__init__()
        self.mha = MultiHeadAttention(d_model, num_heads)
        self.ffn = PositionwiseFeedForward(d_model, d_ff, dropout)
        self.norm1 = nn.LayerNorm(d_model)
        self.norm2 = nn.LayerNorm(d_model)
        self.dropout = nn.Dropout(dropout)
```



이 자료의 모든 저작권은 조치연에게 있습니다 · 무단 도용 금지

Add & Norm을 두 번 반복

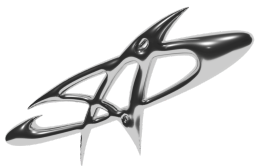
03_encoder_layer_forward.py

```
    def forward(self, x, mask=None):
        # 1) Multi-Head Attention + 잔차 연결 + LayerNorm
        attn_out = self.mha(x, mask)
        x = self.norm1(x + self.dropout(attn_out))

        # 2) Feed Forward + 잔차 연결 + LayerNorm
        ffn_out = self.ffn(x)
        x = self.norm2(x + self.dropout(ffn_out))

        return x
```

x + dropout(sublayer(x)) 패턴이 핵심입니다. 이 한 줄에 Residual Connection, Dropout, Sublayer가 모두 들어 있습니다.



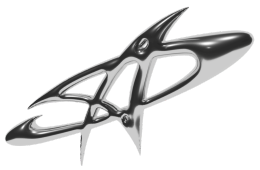
이 자료의 모든 저작권은 조치연에게 있습니다 · 무단 도용 금지

EncoderLayer를 N개 쌓기

04_encoder_stack.py

```
class Encoder(nn.Module):
    def __init__(self, num_layers, d_model, num_heads, d_ff, dropout=0.1):
        super().__init__()
        self.layers = nn.ModuleList([
            EncoderLayer(d_model, num_heads, d_ff, dropout)
            for _ in range(num_layers)
        ])

    def forward(self, x, mask=None):
        for layer in self.layers:
            x = layer(x, mask)   # 이전 layer의 출력이 다음 layer의 입력
        return x
```



이 자료의 모든 저작권은 조치연에게 있습니다 · 무단 도용 금지

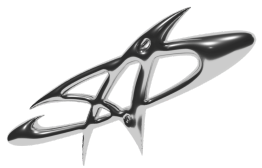
더미 입력으로 forward 테스트

shape이 그대로 유지되는지 확인

- Encoder는 입력과 출력의 **shape이 항상 동일** — [B, T, d_model] → [B, T, d_model]
- 이 성질 덕분에 EncoderLayer를 몇 개든 자유롭게 쌓을 수 있음

```
x = torch.randn(2, 10, 512)   # [batch=2, seq_len=10, d_model=512]
encoder = Encoder(num_layers=6, d_model=512, num_heads=8, d_ff=2048)

out = encoder(x)
print(out.shape)  # torch.Size([2, 10, 512]) — 입력과 동일!
```



이 자료의 모든 저작권은 조치연에게 있습니다 · 무단 도용 금지

5주차 정리

오늘 배운 것 & 다음 주 예고

오늘 배운 것

→Residual: x + Sublayer(x)로 학습 안정화

→Layer Norm: 토큰 단위로 평균0·분산1 정규화

→FFN: 위치별로 독립 적용되는 2층 MLP

→EncoderLayer를 N개 쌓아 Encoder 완성

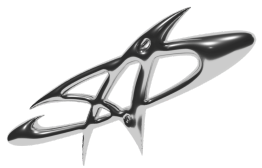
6주차 예고

→Padding Mask와 Look-ahead(Causal) Mask

→Masked Multi-Head Self-Attention

→Encoder-Decoder Cross-Attention

→코딩: DecoderLayer와 Decoder Stack 구현



이 자료의 모든 저작권은 조치연에게 있습니다 · 무단 도용 금지